

Algoritmos e Estruturas de Dados

EN-AEL – 5º Ano

Exercícios de Árvores

Exercício 1)

Recebida uma sequência de N chaves distintas, inseridas numa árvore binária de procura inicialmente vazia. Depois, são recebidas Q consultas. Para cada valor consultado, imprimir a profundidade do nó correspondente na árvore, ou -1 se o valor não existir. No fim, imprimir a travessia **in-order** da árvore.

Entrada

- $N \ Q \rightarrow 10 \ 5$
- N inteiros distintos $\rightarrow [8 \ 3 \ 2 \ 6 \ 9 \ 4 \ 7 \ 1 \ 0 \ 5]$
- Q inteiros, um por consulta $\rightarrow [3 \ 1 \ 4 \ 10 \ 5]$

Saída

- Q números: profundidade de cada chave consultada, ou -1
- última linha: sequência in-order

Exercício 2)

Construir uma BST com N chaves distintas. Depois, receber M operações de remoção. Antes de remover a chave x , imprimir o seu **sucessor in-order** na árvore atual; se não existir, imprimir -1 . Em seguida, remover x .

Entrada

- $N \ M \rightarrow 10 \ 5$
- N inteiros distintos $\rightarrow [8 \ 3 \ 2 \ 6 \ 9 \ 4 \ 7 \ 1 \ 0 \ 5]$
- M inteiros distintos, todos presentes na árvore no momento da remoção $\rightarrow [3 \ 1 \ 4 \ 9 \ 5]$

Saída

- M números, cada um com o sucessor de x antes da remoção, ou -1

Exercício 3)

Numa BST são recebidas N inserções distintas. Após cada inserção, deve-se imprimir:

- a altura atual da árvore; e
- "NOK" se a árvore tiver altura $n-1$ naquele instante, ou "OK" caso contrário.

Nota: uma BST pode degradar-se e comportar-se como uma lista.

Entrada

- $N \rightarrow 10$
- N inteiros distintos $\rightarrow [8\ 3\ 2\ 6\ 9\ 4\ 7\ 1\ 0\ 5]$

Saída

- N pares de valores: altura-estado

Exercício 4)

Inserir N chaves distintas numa AVL vazia. Após cada inserção, imprimir:

- OK, se não foi necessária rotação;
- LL, RR, LR ou RL, consoante o primeiro desequilíbrio resultante da inserção e a corrigir, sendo
 - esquerda-esquerda $\rightarrow$ LL
 - direita-direita $\rightarrow$ RR
 - esquerda-direita $\rightarrow$ LR
 - direita-esquerda $\rightarrow$ RL

Entrada

- $N \rightarrow 10$
- N inteiros distintos $\rightarrow [8\ 3\ 2\ 6\ 9\ 4\ 7\ 1\ 0\ 5]$

Saída

- N valores, um por cada inserção

Exercício 5)

Processar M operações sobre uma AVL inicialmente vazia:

- I - x $\rightarrow$ inserir x
- D - x $\rightarrow$ remover x, se existir

Após cada operação, imprimir o triplo:

- a chave da raiz, ou -1 se a árvore ficar vazia;
- a altura da árvore;

- o número total de rotações simples executadas até ao momento.
Uma rotação dupla conta como 2.

Entrada

- $M \rightarrow 10$
- M pares do tipo I - x ou D - x $\rightarrow [I-8\ I-3\ I-2\ I-6\ I-9\ I-4\ I-7\ D-3\ D-6\ D-2]$

Saída

- M triplos em linha: raiz-altura-rotacoes

Exercício 6)

É dada a descrição de uma árvore binária Red-Black em que cada nó tem:

- chave;
- cor (R ou B);
- índice do filho esquerdo;
- índice do filho direito.

Determinar se a árvore é uma **Red-Black válida**. Se for válida, imprimir a black-height da raiz. Caso contrário, imprimir NOK.

Entrada

- N
- N quadras: chave-cor-esq-dta
- índice da raiz

Saída

- OK-h ou NOK

Entrada:

```
10
20-B-2-3
10-B-4-5
30-B-6-7
5-B-8-0
15-B-0-9
25-B-0-10
35-B-0-0
2-R-0-0
17-R-0-0
27-R-0-0
1
```

Resolução

Exercício 1)

Construir a BST por inserção direta. Cada inserção segue a regra:

- menor que o nó atual → esquerda
- maior que o nó atual → direita

Depois, para cada consulta, faz-se procura iterativa acumulando a profundidade. No fim, uma DFS in-order produz a lista ordenada, porque numa BST a travessia (Esquerda, Próprio, Direita) devolve as chaves por ordem crescente.

Saída:

[3 1 4 10 5]- → [1 3 3 -1 4]

[0 1 2 3 4 5 6 7 8 9]

Detalhe:

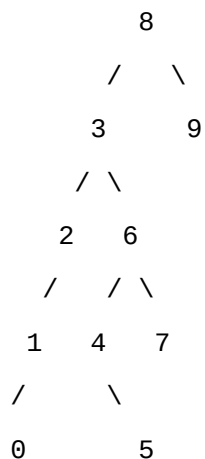
Assumindo que as chaves inseridas na BST são:

8 3 2 6 9 4 7 1 0 5

e as consultas são:

3 1 4 10 5

a árvore fica:



Profundidades

Considerando a raiz com profundidade 0:

- 3 → profundidade 1
- 1 → profundidade 3
- 4 → profundidade 3
- 10 → não existe → -1
- 5 → profundidade 4

Travessia in-order

Numa BST, a travessia **in-order** devolve os valores por ordem crescente:

0 1 2 3 4 5 6 7 8 9

Saída

1 3 3 -1 4

0 1 2 3 4 5 6 7 8 9

Complexidade

- Construção: $O(N \cdot h)$
- Consultas: $O(Q \cdot h)$
- In-order: $O(N)$
- No pior caso, $h = N$, se a árvore degenerar.

Exercício 2)

Há duas tarefas:

1. encontrar o sucessor de x ;
2. remover x preservando a propriedade da BST.

O sucessor de um nó:

- se tiver subárvore direita, é o **mínimo da subárvore direita**;
- caso contrário, sobe-se pelos pais até encontrar o primeiro ancestral para o qual o nó está à esquerda.

A remoção tem os três casos clássicos:

- nó folha;
- nó com um filho;
- nó com dois filhos, substituindo pelo sucessor.

No material, a remoção da BST é precisamente apresentada por estes casos, com destaque para o “mínimo da sub-BST da direita”.

[4 2 5 -1 6]

Sim. Com a correção **10** → **9**, a entrada passa a ficar consistente.

Dados

- $N \ M = 10 \ 5$
- chaves da BST:
[8, 3, 2, 6, 9, 4, 7, 1, 0, 5]
- remoções:
[3, 1, 4, 9, 5]

Ordem in-order inicial

0 1 2 3 4 5 6 7 8 9

Agora, antes de cada remoção, tomamos o **sucessor in-order** de x , isto é, o menor valor maior que x na árvore atual.

1) Remover 3

Sucessor de 3: 4

2) Remover 1

Árvore atual em ordem:

0 1 2 4 5 6 7 8 9

Sucessor de 1: 2

3) Remover 4

Árvore atual em ordem:

0 2 4 5 6 7 8 9

Sucessor de 4: 5

4) Remover 9

Árvore atual em ordem:

0 2 5 6 7 8 9

9 é o maior elemento, portanto: -1

5) Remover 5

Árvore atual em ordem:

0 2 5 6 7 8

Sucessor de 5: 6

Saída

[4, 2, 5, -1, 6]

Portanto, a solução correta é:

[4, 2, 5, -1, 6]

Complexidade

- Cada sucessor/remoção: $O(h)$
- Total: $O(M \cdot h)$

Exercício 3)

Simular a BST normalmente e, após cada inserção, atualizar altura ao subir pelos pais. A árvore está degenerada (NOK) quando a altura é igual a $n - 1$, isto é, quando existe apenas um caminho vertical. Este problema explora diretamente o caso extremo: uma BST construída a partir de valores ordenados pode perder eficiência e aproximar-se de uma lista, passando a ter custo temporal linear.

Saída:

0-NOK 1-NOK 2-NOK 2-OK 2-OK 3-OK 3-OK 3-OK 4-OK 4-OK

Assumindo a definição usual neste contexto:

- **altura da árvore** = número de **arestas/ligações** no caminho mais longo da raiz até uma folha;
- logo, uma árvore com n nós está **degenerada** quando a altura é $n - 1$.

Inserções na BST

Sequência:

8, 3, 2, 6, 9, 4, 7, 1, 0, 5

Evolução

1) Inserir 8

Árvore:

8

Altura = 0

Como $n = 1$, então $n - 1 = 0 \rightarrow \text{NOK}$

2) Inserir 3

Árvore:

8

/

3

Altura = 1

Como $n = 2$, então $n - 1 = 1 \rightarrow \text{NOK}$

3) Inserir 2

Árvore:

8

/

3

/

2

Altura = 2

Como $n = 3$, então $n - 1 = 2 \rightarrow \text{NOK}$

4) Inserir 6

Árvore:

8

/

3

/ \

2 6

Altura = 2

Como $n = 4$, então $n - 1 = 3 \rightarrow \text{OK}$

5) Inserir 9

Árvore:

8

/ \

3 9

/ \

2 6

Altura = 2

Como $n = 5$, então $n - 1 = 4 \rightarrow \text{OK}$

6) Inserir 4

Árvore:

8

/ \

3 9

/ \

2 6

/

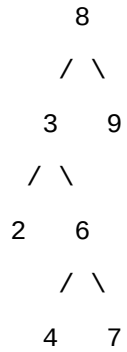
4

Altura = 3

Como $n = 6$, então $n - 1 = 5 \rightarrow \mathbf{OK}$

7) Inserir 7

Árvore:

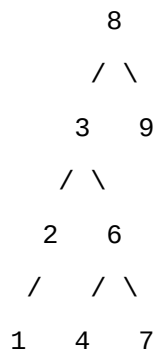


Altura = 3

Como $n = 7$, então $n - 1 = 6 \rightarrow \mathbf{OK}$

8) Inserir 1

Árvore:

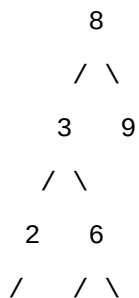


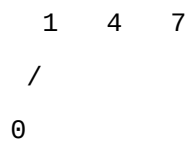
Altura = 3

Como $n = 8$, então $n - 1 = 7 \rightarrow \mathbf{OK}$

9) Inserir 0

Árvore:



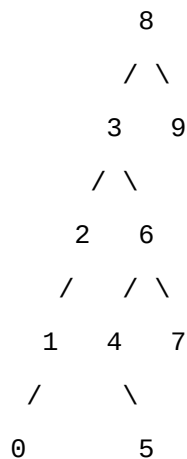


Altura = 4

Como $n = 9$, então $n - 1 = 8 \rightarrow \mathbf{OK}$

10) Inserir 5

Árvore:



Altura = 4

Como $n = 10$, então $n - 1 = 9 \rightarrow \mathbf{OK}$

Saída

0-NOK 1-NOK 2-NOK 2-OK 2-OK 3-OK 3-OK 3-OK 4-OK 4-OK

Complexidade

- Inserção: $O(h)$ por operação
- Atualização local da altura: $O(h)$
- Total: $O(N \cdot h)$

Exercício 4)

Inserir como numa BST. Depois, subir desde o pai do novo nó até à raiz, recalculando alturas. Ao encontrar o primeiro ancestral com $|\text{balance}| > 1$, identificar o caso: “nós em linha” versus “triângulo”, com rotação simples ou dupla.

Entrada

- $N \rightarrow 10$
- N inteiros distintos $\rightarrow [8\ 3\ 2\ 6\ 9\ 4\ 7\ 1\ 0\ 5]$

Saída:

[OK, OK, LL, OK, OK, RL, OK, OK, LL, OK]

Justificação, inserção a inserção

Chaves:

[8, 3, 2, 6, 9, 4, 7, 1, 0, 5]

1) Inserir 8

Árvore com um só nó.

Sem desequilíbrio $\rightarrow$ OK

2) Inserir 3

3 fica à esquerda de 8.

Ainda balanceada $\rightarrow$ OK

3) Inserir 2

Fica o caminho $8 \rightarrow 3 \rightarrow 2$, isto é, caso **esquerda-esquerda** no nó 8.

Rotação simples à direita $\rightarrow$ LL

4) Inserir 6

Entra sem provocar desequilíbrio $\rightarrow$ OK

5) Inserir 9

Entra sem provocar desequilíbrio $\rightarrow$ OK

6) Inserir 4

O primeiro desequilíbrio surge com configuração **direita-esquerda**.

Rotação dupla $\rightarrow$ RL

7) Inserir 7

A árvore mantém-se balanceada $\rightarrow$ OK

8) Inserir 1

Sem necessidade de rotação → OK

9) Inserir 0

Surge desequilíbrio **esquerda-esquerda** no primeiro ancestral crítico.

Rotação simples à direita → LL

10) Inserir 5

Não há desequilíbrio → OK

Saída

Em formato de lista:

[OK, OK, LL, OK, OK, RL, OK, OK, LL, OK]

Complexidade

- $O(\log N)$ amortizado por inserção numa AVL
- Total: $O(N \log N)$

Exercício 5)

Manter uma implementação completa de AVL com:

- inserção BST;
- remoção BST;
- atualização de altura;
- rebalanceamento a subir.

Na remoção, além dos três casos da BST, pode ser necessário reequilibrar vários ancestrais até à raiz. Isso aparece explicitamente nos slides da remoção em AVL.

Saída:

[8-0-0, 8-1-0, 3-1-1, 3-2-1, 3-2-1, 6-2-3, 6-2-3, 6-2-3, 7-2-3, 7-2-3]

Passo a passo

Operações:

[I-8, I-3, I-2, I-6, I-9, I-4, I-7, D-3, D-6, D-2]

1) I-8

AVL: só tem a raiz 8

Triplo:

3?

Corrigindo: a raiz é 8, altura 0, rotações 0

8-0-0

2) I-3

Árvore:

8

/

3

Sem rotação:

8-1-0

3) I-2

Fica um caso **LL** em 8, faz-se uma rotação simples à direita.

Nova árvore:

3

/ \

2 8

Triplo:

3-1-1

4) I-6

Entra como filho esquerdo de 8:

```
      3
     / \
    2   8
     /
    6
```

Sem nova rotação:

3-2-1

5) I-9

```
      3
     / \
    2   8
     / \
    6   9
```

Ainda balanceada:

3-2-1

6) I-4

Entra em 6 à esquerda, criando um caso **RL** no nó 3:

- rotação direita em 8
- rotação esquerda em 3

Duas rotações simples no total.

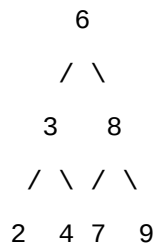
Nova árvore:

```
      6
     / \
    3   8
   / \   \
  2  4   9
```

Triplo:

6-2-3

7) I-7



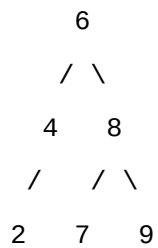
Sem rotação:

6-2-3

8) D-3

Remove-se 3 usando o sucessor 4.

Fica:



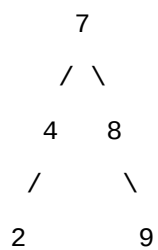
Ainda balanceada:

6-2-3

9) D-6

Remove-se 6 usando o sucessor 7.

Fica:



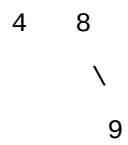
Sem rotação:

7-2-3

10) D-2

Fica:





Ainda balanceada:

7-2-3

Saída

8-0-0 8-1-0 3-1-1 3-2-1 3-2-1 6-2-3 6-2-3 6-2-3 7-2-3 7-2-3

Complexidade

- Cada operação: $O(\log N)$
- Total: $O(M \log N)$

Exercício 6)

Fazer uma procura na árvore que verifique simultaneamente:

1. propriedade de BST;
2. raiz black;
3. se um nó é red, então os filhos reais são black;
4. todas as rotas até NIL têm a mesma black-height.

A procura na árvore devolve:

- validade;
- black-height.

Se qualquer subárvore falhar, a árvore é inválida.

Entrada:

10

20-B-2-3

10-B-4-5

30-B-6-7

5-B-8-0

15-B-0-9

25-B-0-10

35-B-0-0

2-R-0-0

17-R-0-0

27-R-0-0

1

Leitura dos nós

Cada linha tem o formato:

chave-cor-esq-dta

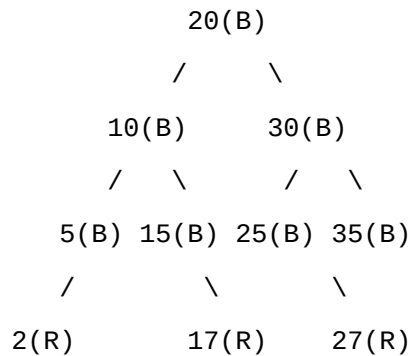
Logo:

- nó 1: 20-B-2-3
- nó 2: 10-B-4-5
- nó 3: 30-B-6-7
- nó 4: 5-B-8-0
- nó 5: 15-B-0-9
- nó 6: 25-B-0-10

- nó 7: 35-B-0-0
- nó 8: 2-R-0-0
- nó 9: 17-R-0-0
- nó 10: 27-R-0-0

raiz = índice 1

Árvore



Verificação

1) Propriedade de BST

Para cada nó:

- esquerda < nó
- direita > nó

Verificando:

- $10 < 20 < 30$
- $5 < 10 < 15$
- $25 < 30 < 35$
- $2 < 5$
- $17 > 15$
- $27 > 25$

Logo, a propriedade de BST é satisfeita.

2) A raiz é black

A raiz é 20(B).

Logo, válida nesta condição.

3) Nenhum nó red tem filho red

Os nós red são:

- 2(R)

- 17(R)
- 27(R)

Todos eles têm filhos NIL, que são considerados **black**.

Logo, **não há dois reds consecutivos**.

4) Todos os caminhos até NIL têm o mesmo número de nós black

Vamos contar a **black-height da raiz**.

Exemplos de caminhos:

- 20 → 10 → 5 → 2 → NIL
- 20 → 10 → 5 → NIL
- 20 → 10 → 15 → 17 → NIL
- 20 → 30 → 25 → 27 → NIL
- 20 → 30 → 35 → NIL

Sem contar a raiz 20, e contando NIL, o número de nós black em cada caminho é sempre:

3

Por exemplo:

- 20 → 10 → 5 → 2 → NIL
blacks: 10, 5, NIL → 3
- 20 → 30 → 35 → NIL
blacks: 30, 35, NIL → 3

Logo, a árvore é uma **Red-Black válida** e a **OK-height da raiz é 3**.

Saída

OK-3

Complexidade

- $O(N)$